LLM-based Agent 综述

陈凌锐

2026 3.18

基本概念&引言

•LLM-based Agent 的新定义：以大语言模型为认知核心，集成感知、规划、记忆、行动等模块，能在复杂环境中自主完成任务的系统

➢相对于LLM：

传统 LLM 使用方式：

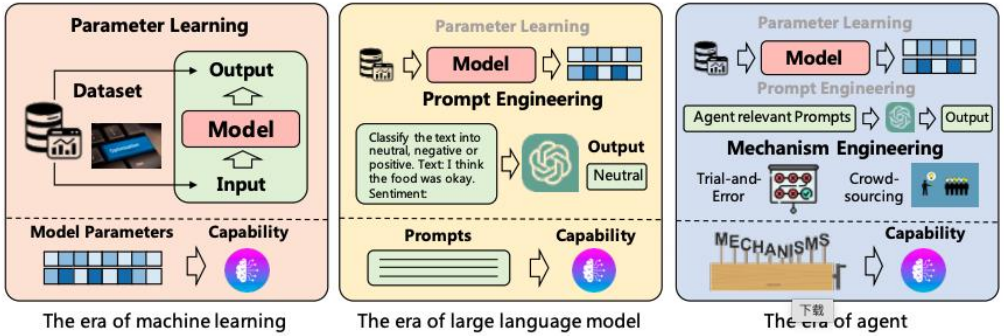
Human → Prompt → LLM → Response → Human

❌ 单轮输入输出，无法感知外部环境，无法自主行动

LLM-based Agent：

Perceive → Think → Act → Observe → Think → Act → ...

✅ 自主循环，持续交互，闭环决策



➢ 相对于传统agent：以前的 Agent（如 BDI Agent、RL Agent）需要针对特定任务从头设计/训练，LLM 提供了前所未有的通用性 + 语言接口

➢ 相关文献

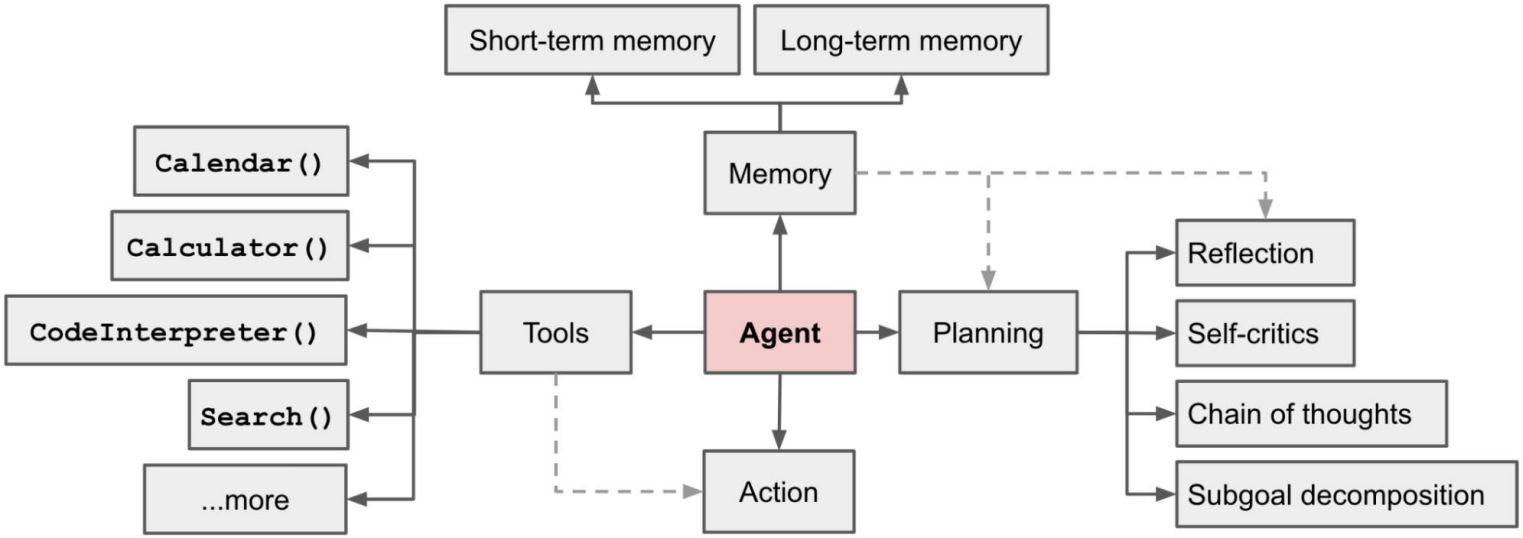
➢ Lilian Weng Blog: "LLM Powered Autonomous Agents" (2023)

➢ Wang et al., "A Survey on Large Language Model based Autonomous Agents" (2024)

➢ Xi et al., "The Rise and Potential of Large Language Model Based Agents: A Survey" (2023)

整体框架

- planning memory
- toolcalling action
- muti-agent workflow



规划 planning

- planning存在的作用是：
- 把LLM无法一次性完成或一次性完成效果差的复杂任务，按照一定逻辑规则。拆分成更小、更简单、更易于管理的子目标，并设计不同的流程来确保目标的达成度。

- 常用拆分执行逻辑
  - without feekback 一次性生成计划/推理链，不依赖环境反馈
  - with feedback 根据执行结果、环境反馈、自我反思来动态调整

规划 planning

- 思维链（CoT。2022）
- 在prompt明确指示模型“逐步思考”，从而利用更多的测试时间进行计算，将复杂任务分解为更小、更简单的步骤。CoT 将大型任务转化为多个易于处理的任务，并有助于理解模型的思维过程。

- 思维树（ToT，2023）扩展了思维导图（CoT），在每个步骤中探索多种推理可能性。它首先将问题分解为多个思维步骤，并在每个步骤中生成多个想法，从而构建树状结构。搜索过程可以是广度优先搜索（BFS）或深度优先搜索（DFS），每个状态都由分类器（通过提示）或多数投票进行评估。

- Current State: 需要解决"24点游戏：用4 5 6 10算出24"
- [调用LLM] 生成k个候选思维（k=3-5）：

- Thought 1: 先算 10 - 6 = 4
- Thought 2: 先算 6 × 4 = 24
- Thought 3: 先算 5 + 4 = 9

```
CoT:   问题 → 步骤1 → 步骤2 → ... → 答案   (单链)

ToT:   问题 → ┬ 想法A → ┬ 展开...
              │         └ 展开...
              ├ 想法B → 展开... ✓ 选中
              └ 想法C → 剪枝 ✗
```

规划 planning Rexxx

上述策略先让 LLM 生成完整计划，再按计划逐步执行，有着明显的两阶段分离：规划阶段 vs 执行阶段

而另一种**feedback工作流**则根据执行结果、环境反馈、自我反思来动态调整，action与planning交替进行

- ReAct —— 推理与行动交织, ICLR 2023
- Reflexion —— 自我反思 NeurIPS 2023
- CoH—— Chain of Hindsight 2023

规划 planning Rexxx

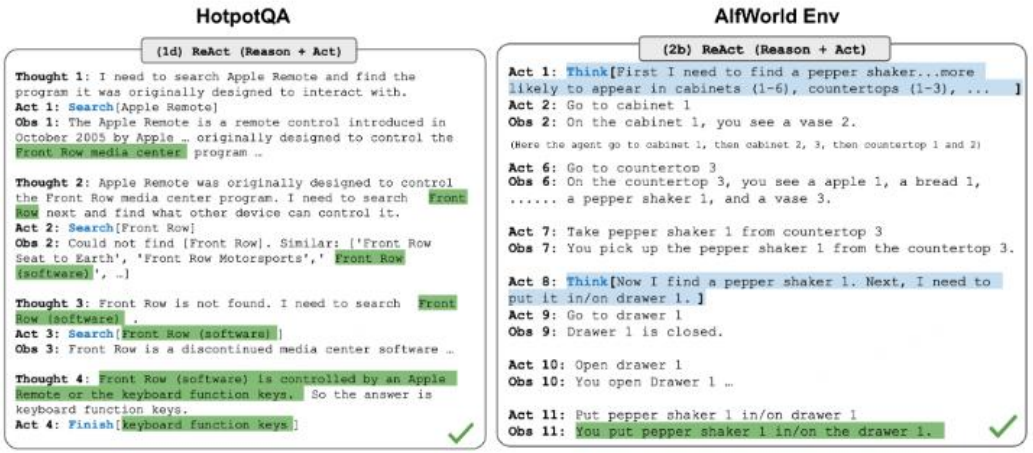
ReAct（ICLR,2023）通过将动作空间扩展为特定任务的离散动作和语言空间的组合，将推理和行动整合到语言学习模型（LLM）中。

前者使LLM能够与环境交互（例如使用维基百科搜索API），而后者则促使LLM生成自然语言的推理轨迹。

核心思想： Reasoning + Acting 交替进行，每一步都**先推理再行动**，没有独立的反思环节

ReAct 提示模板包含了供 LLM 思考的明确步骤

```
Thought: ...
Action: ...
Observation: ...
... (Repeated many times)
```



规划 planning Rexxx

ReAct：tao循环

1. 推理（Thought）：Agent将目标、当前状态、历史交互记录拼入Prompt，LLM生成一段推理，分析当前该做什么、为什么。
2. 行动（Action）
    - LLM输出一个具体的工具调用指令：
    - Thought: 需要搜索"ReAct论文作者"来回答用户问题。Action: search("ReAct paper authors")
3. 观察（Observation）
    - 执行器调用工具，返回结果，直接追加到上下文中：
    - Observation: ReAct由Shunyu Yao等人于2022年提出...
4. 继续循环或结束
    - 将完整历史 T1→A1→O1→T2→A2→O2→... 全部拼入Prompt，LLM决定：
    - 继续： 生成下一轮 Thought → Action
    - 结束： 直接输出 Final Answer

交互模式总结：

Thought → Action → Observation → Thought → Action → Observation → ... → Final Answer

单层循环，所有历史线性拼接进Prompt

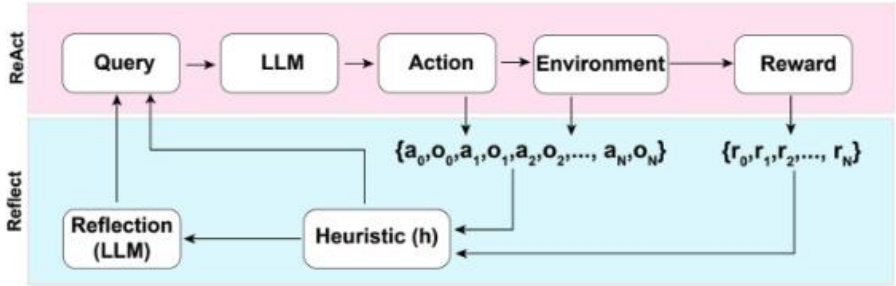
LLM 每次只决定一步，不做事后反思

如果中间出错，只能在下一个Thought中"隐式"调整，没有专门的反思机制

规划 planning Rexxx

Reflexion（2023）旨在赋予智能体动态记忆和自我反思能力，以提升其推理技能。**Reflexion采用标准的强化学习（RL）设置，其中奖励模型提供简单的二元奖励**，动作空间则遵循ReAct的设置，ReAct在特定任务的动作空间中添加了语言信息，以支持复杂的推理步骤。**每次动作之后，智能体都会进行自我反思**。

根据自我反思的结果，还可以选择重置环境以开始新的试验。



内层循环（单次尝试 Episode）

和ReAct基本一致：

Thought → Action → Observation → … → 得到最终结果

规划 planning Rexxx

外层循环（跨尝试的反思学习）

1. 评估（Evaluate）：**单次尝试结束后，通过外部评估器或LLM自身判断结果是否正确/达标**：
2. 反思（Reflect）— Reflexion的核心

若失败，Agent调用LLM（可以是同一个，也可以是专门的Reflector）生成一段自然语言的经验总结：

Reflection: "上一次我直接搜索了完整问题，结果太泛。下次应该xxx。另外我忽略了xxxx。"

3. 存入记忆（Memory Update）这段Reflection文本被存入长期记忆。
4. 重新尝试（Retry）发起新一轮Episode，Prompt中额外注入历次反思记录：

[系统指令 + 目标]

[Previous Reflections]:

- Trial 1: "不应该直接搜索完整问题..."
- Trial 2: "子问题拆解有效，但忘记验证来源..."

[开始新的 Thought → Action → Observation 循环]

5. 循环直到成功或达到最大尝试次数

交互模式总结：

```
· Episode 1: Thought→Action→Obs→...→结果 ✗
  └ Reflect → 生成经验总结 → 存入Memory
· Episode 2: [带上Reflection] Thought→Action→Obs→...→结果 ✗
  └ Reflect → 更新Memory
· Episode 3: [带上所有Reflection] → 结果 ✓ 完成!
```

双层循环：内层是ReAct式执行，外层是反思学习

两种LLM调用：Actor（执行任务）+ Reflector（生成反思）

反思以自然语言存储，作为下次尝试的额外上下文注入Prompt

与CoH的区别：Reflexion的反思发生在整个Episode结束后，而非每一步中

规划 planning Rexxx

CoH是CoT的变体，更侧重于事后反思和从经验中学习。Agent在执行一系列行动后，会**回顾整个过程**，思考哪些行动是有效的，哪些是无效的，以及为什么，并从中提取经验教训，用于指导未来的规划。

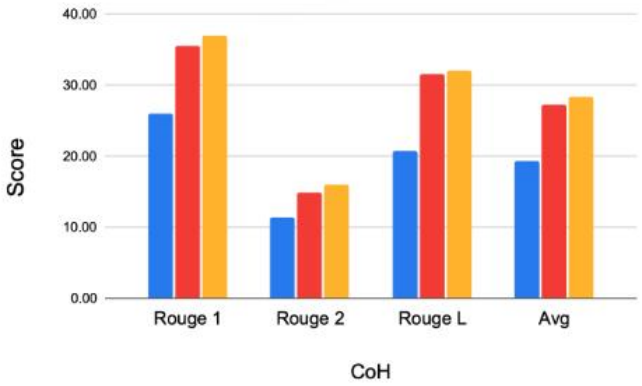
Prompt构造（输入LLM）：
Agent向LLM发送Prompt，包含三要素：目标（Goal）、当前状态（Current State）、可用工具/可行动列表（Available Tools/Actions），以及明确的指令要求LLM“一步步思考”（如“Let's think step by step”或“Explain your reasoning”）。

思考&行动：
LLM在Prompt驱动下生成一段“思考链”输出。随后LLM输出一个具体的行动（Action），常为结构化命令，例如：

Thought: Need to query weather before scheduling. Use get_current_weather tool.Action: {"tool_name": "get_current_weather", "parameters": {"location": "London"}}
Agent的执行器解析Action并调用对应工具或内部操作。

观察：
工具返回观察结果（Observation），如API结果、操作成功/失败信息等；Agent将此加入“当前状态”。

反思（CoH）：
Agent把最新**Observation连同历次思考/行动/观察历史一起回传给LLM，继续下一轮规划。如果结果异常或失败，会增加明确的反思提示**，如“为什么失败？出了什么问题？如何调整？能总结什么经验？”—以帮助Agent从失败中修正策略并重新规划。



memory

- Memory
- 针对的是LLM 的 context window 有限，长期交互中信息会"遗忘"（滑出窗口）需要从海量历史中选择性提取和本次问答”相关”的信息以及解决参数知识静态的问题：训练截止后无法更新知识
- 下图是人类的记忆模式归类，agent基于此进行了模仿

- **Memory**
  - Sensory memory
    - Iconic memory (visual)
    - Echoic memory (auditory)
    - Haptic memory (touch)
  - Short-term memory (Working memory)
  - Long-term memory
    - Explicit / Declarative memory (conscious)
      - Episodic memory (life events)
      - Semantic memory (facts, concepts)
    - Implicit / Procedural memory (unconscious; skills)

memory

- **Memory**
  - Sensory memory
    - Iconic memory (visual)
    - Echoic memory (auditory)
    - Haptic memory (touch)
  - Short-term memory (Working memory)
  - Long-term memory
    - Explicit / Declarative memory (conscious)
      - Episodic memory (life events)
      - Semantic memory (facts, concepts)
    - Implicit / Procedural memory (unconscious; skills)

➢我们可以大致考虑以下几种映射关系：

- 感觉记忆是学习嵌入原始输入（包括文本、图像或其他模态）的表征；（本次问答原始输入）
- 形式：原始文本输入、图像像素、音频波形、API 返回的 JSON 等
- 操作：感知编码（Perception Encoding）和注意力过滤（Attention Filtering）

- 短期记忆Agent 在当前任务执行过程中活跃使用的信息，直接对应 LLM 的上下文窗口（Context Window）
- LLM 本身是无状态的。 它不会"记住"上一轮说了什么。每次推理都是一次全新的、独立的前向传播
- 它很短且有限，因为它受到Transformer有限上下文窗口长度的限制。
- 操作：截断 压缩 转入长期记忆

➢长期记忆作为外部向量存储，代理可以在查询时关注它，需要时检索回来，注入上下文窗口 。（外挂知识库等）（重点）

memory

长期记忆是 Memory 模块研究的核心战场，又分为三类：

子类型	含义	Agent 中的对应	典型实现
情景记忆 (Episodic)	过去发生过的具体事件	Agent 的行为日志、对话记录	时间戳+事件描述存入向量库
语义记忆 (Semantic)	抽象的事实性知识	外部知识库、文档	RAG、知识图谱
程序记忆 (Procedural)	如何执行某任务的技能	工具调用模板、成功 plan 的复用	Skill Library (Voyager)

Tool

不论哪一种，大家关心的操作无非三类：

- 记忆编码 (write)：什么信息值得记住？以什么形式存储？
- 记忆检索 （read）：面对当前任务/问题，如何从海量记忆中精准取回最有用的信息？
- 记忆管理（reflection）：记忆越来越多，如何保持记忆库的质量、避免过载？

memory-编码

- 记忆编码 (write)：什么信息值得记住？以什么形式存储？

编码方式	适用场景	优点	缺点
Embedding 向量化	非结构化文本（RAG）	语义检索能力强	不擅长精确匹配
知识图谱三元组	结构化事实知识	推理路径清晰	构建成本高
自然语言摘要	对话历史压缩	人类可读，灵活	摘要可能丢失细节
原始文本+元数据	日志记录	信息无损	检索效率低

```python
# 伪代码示意
def memory_write(new_experience):
    # 1. 信息提取：从原始交互中抽取关键信息
    key_info = extract(new_experience)

    # 2. 编码：转化为存储格式
    encoded = encode(key_info)  # text / embedding / KG triple

    # 3. 去重与合并：避免冗余
    if similar_memory_exists(encoded):
        merge(existing_memory, encoded)
    else:
        store(encoded)

    # 4. 重要性评分
    importance = score_importance(key_info)
    add_metadata(encoded, importance=importance, timestamp=now())
```

memory-检索

- 记忆检索 （read）：面对当前任务/问题，如何从海量记忆中精准取回最有用的信息？

$$m^* = \arg\max_{m \in M} \left(\alpha s^{rec}(q,m) + \beta s^{rel}(q,m) + \gamma s^{imp}(m) \right),$$

- M:所有记忆集合
- rec：时效性 $recency(m) = \gamma^{t_{now} - t_m}$
- rel:相关性 $relevance(q,m) = \frac{\mathbf{e}_q \cdot \mathbf{e}_m}{\|\mathbf{e}_q\| \cdot \|\mathbf{e}_m\|}$
- imp: 重要性 Generative Agents 的做法：直接让 LLM 打分！
- 提取出的相关信息则根据top-n，被加入到模型的context
- 注意：重要性与q无关，查询时simp是没有q参数的

- 其他方法：Reranking（"给定当前问题 Q，以下哪些记忆最相关？请排序"），
  - 纯向量检索（最简单，很多系统用这种）

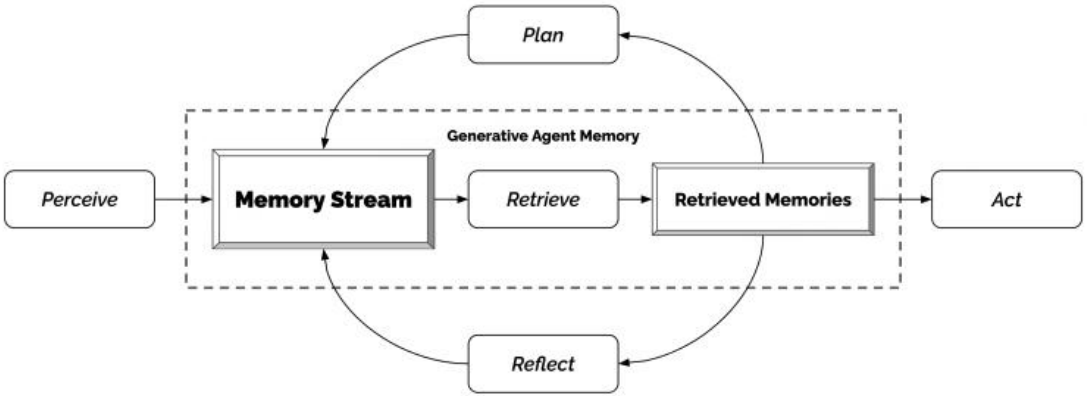
memory-管理与反思

- 去重和合并：记忆聚合、重复条目合并，或者把多条相似经验压缩成一条更通用的记录。这既节省空间，也提升检索质量。
- 遗忘策略。基于 FIFO 删最旧的，基于重要性删最不重要的，或者基于时间衰减淘汰长期不用的记忆。
- 知识更新。语义相近的旧条目，如果满足一些规则判断，比如同一个实体、同一个属性位上出现不同值，就认为冲突，可以做旧记忆失效，新记忆替代旧记忆。更稳妥的是版本化
- 压缩。保留关键信息，减少 token 和存储负担。可以通过LLM 摘要，结构化压缩等
- 组织结构优化：随着记忆增加，系统可能要重新建索引、重建向量库、按主题分桶、建立层级结构或 key-value 结构，让后续检索更高效。
- **错的记得住，对的记不住。主要压缩掉做得好的**

- 反思机制

memory-管理与反思

- "Generative Agents: Interactive Simulacra of Human Behavior" — Park et al., 2023



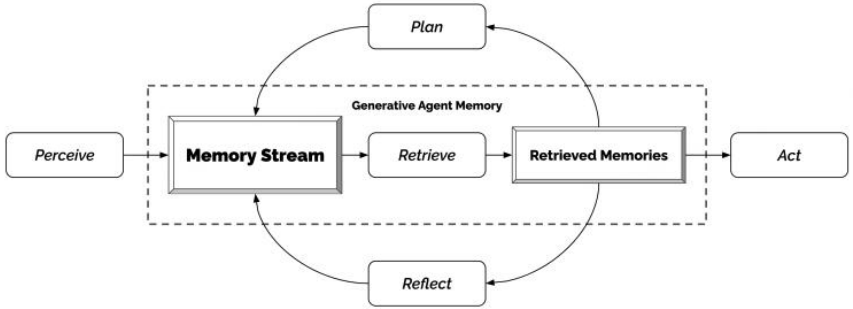
- 在小镇中，每个 Agent 都有一个“记忆流”。这是一个包含了所有过去经历的按时间排序的列表。

memory-管理与反思

1.观察（Observation）： Agent 每一刻都在感知环境，比如“John 走进咖啡馆看到 Alice”。
2.存入记忆流： 每一个观察被打包成一个“记忆对象”，包含：

- 内容（Content）： 描述具体发生了什么。
- 时间戳（Timestamp）： 创建时间。
- 最后访问时间（Last Access）： 用于计算时效性。
- 重要性评分（Importance）： 由 LLM 自动打分

检索（Retrieve）： 当 Agent 需要做决定时，它会拿着当前任务（Query）去检索记忆流。
◦公式： Score=α· Recency+β· Importance+γ· RelevanceScore=α· Recency+β· Importance+γ· Relevance。
◦Agent 会取出得分最高的 Top-K 记忆，放入 Prompt 中作为上下文，让 LLM 基于这些“回忆”做出决定。



memory-管理与反思

“反思”是该论文最精妙的设计，它解决了记忆碎片化的问题。如果没有反思，Agent 永远只能看到零散的观察（比如“John 昨天喝咖啡”、“John 前天喝咖啡”），而无法理解更深层的动机。

动量更新

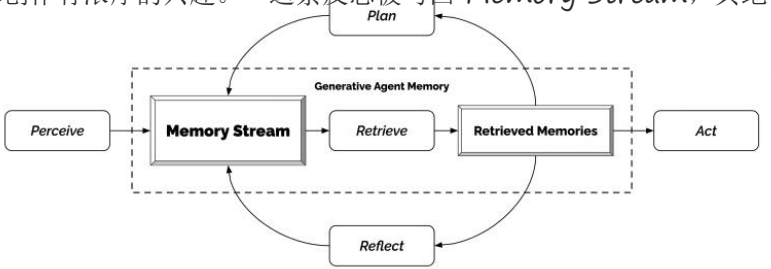
1.触发条件： 当 Agent 感知到的“重要性”累积值达到一定阈值（例如 150 分）时，系统就会自动触发反思。

2.设计逻辑： 这模拟了人类在处理完大量琐事后，静下来思考“这对我意味着什么”的过程。

- 第一步：提问（Identify Questions）。
- Agent 检索最近的记忆流，问自己：“最近这些记忆里，有什么值得注意的主题或问题？”
    - 例如：John 看到自己的记忆流里有“买了画具”、“在画廊待了很久”、“讨论莫奈”。LLM 会生成问题：“John 最近对什么感兴趣？”
- 第二步：检索（Query Memory）。
    - Agent 用生成的这些问题去检索相关的原始记忆。
- 第三步：总结（Generate Insights）。
    - LLM 对检索到的这些原始记忆进行综合分析，写下一条新的“反思记录”。
    - 例如：LLM 得出结论：“John 对艺术创作有浓厚的兴趣。” 这条反思被写回 Memory Stream，其地位等同于原始观察，且通常被赋予更高的重要性分数。

•没有反思时： Agent 仅仅是基于“我昨天去过画廊”这个行为去行动。

•有了反思后： Agent 会检索到“我是一个热爱艺术的人”这一抽象认知，于是它可能会主动去查询画展信息、主动去买画具。

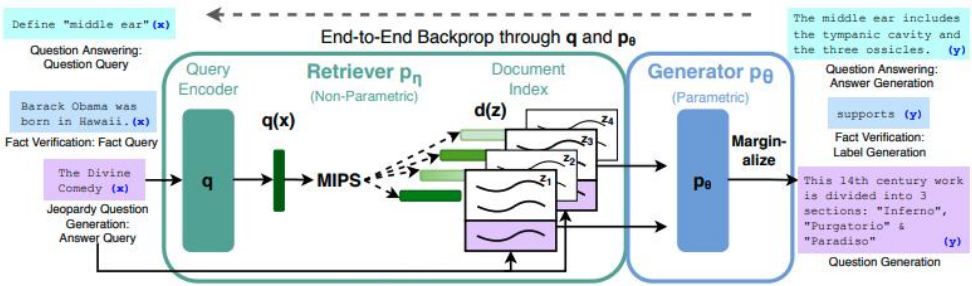


memory-rag

- 论文："Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"（Lewis et al., NeurIPS 2020）

- RAG（Retrieval-Augmented Generation，检索增强生成），简单说就是：
- 在大模型生成回答之前，先去外部知识源里检索相关信息，再把检索结果连同问题一起喂给模型生成答案。
- 如果把 Agent 的 Memory 分成短期记忆和长期记忆，RAG 对应的就是长期记忆中的语义记忆。

- 离线阶段（建库）：
- 文档 → 切片(Chunking) → 向量化(Embedding) → 存入向量数据库（高维数组）
- 在线阶段（查询）：
- 用户问题 → 向量化 → 从向量库检索相似片段（找到原文字） → 拼进 prompt → 调 LLM → 回答



工作结合

Agent 模块	Overcooked 中的映射	解决的核心问题
Planning（规划）	菜谱分解、子任务序列生成（取食材→切→煮→装盘→送餐）	长视野任务需要多步骤有序完成，单次 LLM 调用无法一步到位
Memory（记忆）	短期：当前厨房状态/队友位置/锅的状态；长期：历史成功协作模式、失败经验	上下文窗口有限，无法记住全局历史；需要跨 episode 学习
Tool Calling（工具调用）	LLM 输出的高层决策 → 调用 RL 底层策略执行具体动作	LLM 不直接控制网格移动，需要一个"执行器"来转化语言决策为环境动作